

BABA LAGIN

REST API Specification

Backend integration reference for the registration, payment, authentication and admin APIs

Version	1.0
Date	June 27, 2026
Application	loginapp (Spring Boot 2.7.18, Java 8)
Base URL	http://localhost:8080
Media type	application/json (unless noted)

1. Overview

This document specifies all REST APIs exposed by the loginapp backend that powers the BABA LAGIN matrimonial web application. The frontend is a React single-page application served by the same Spring Boot server, so all endpoints are same-origin under the base URL.

There are four groups of endpoints:

- Registration : submit and read matrimonial enroll-form data.
- Payment : submit a payment with a screenshot, and (admin) verify/reject it.
- Authentication : login (admin + profile), session token, idle timeout.
- Admin reads : list/inspect registrations and payments.

Persistence: MySQL (default profile) or H2 (dev profile). Tables auto-create on startup.

Note: Replace localhost:8080 with the deployed host/port when integrating.

2. Authentication

Authentication is token-based. A login endpoint validates credentials against the users / registrations table and issues an opaque server-side session token. The token is sent on subsequent requests in the X-Auth-Token header and is validated on the server. Sessions carry a 30-minute idle (inactivity) timeout and are held in memory, so a server restart also ends every session. (HTTP Basic is no longer used.)

Admin login

POST /api/admin/login with a JSON body of username + password. A default admin account is seeded on first startup:

Username	Password	Role
admin	admin123	ROLE_ADMIN

Override the seeded credentials via app.admin.username / app.admin.password in application.properties.

```
POST /api/admin/login
Content-Type: application/json

{ "username": "admin", "password": "admin123" }

200 OK
{ "token": "9f2c...32hex", "username": "admin",
  "roles": ["ROLE_ADMIN"], "instanceId": "<server-instance-uuid>" }

401 = bad credentials, 403 = not an administrator.
```

Profile login

POST /api/login (Registration ID + email) also returns a token field alongside the profile data. Use it the same way for the logged-in profile session.

Sending the token

Send the X-Auth-Token header with every request to a protected endpoint (and on the session endpoints below):

```
X-Auth-Token: 9f2c1a7b4d... (the token returned by login)
```

Session lifetime (30-minute idle timeout)

The server tracks each session's last-activity time. Two endpoints manage the sliding window:

Endpoint	Purpose
----------	---------

GET /api/session/validate	Read-only liveness check. 200 if live, 401 once idle-expired or after a restart. Does NOT extend the window (so polling can't keep a dead session alive). The client polls this and logs out on 401.
POST /api/session/heartbeat	Records activity and extends the idle window by 30 min. The client calls it on real user interaction (throttled). 401 if already expired.
POST /api/session/logout	Invalidates the current session token (used by the profile logout). Idempotent (204).
POST /api/admin/logout	Invalidates the admin token immediately.

Both endpoints take the X-Auth-Token header. A session with no activity (no heartbeat) for 30 minutes is rejected on the next validate / protected request, and the client logs the user out.

Access levels

Level	Requirement	Endpoints
Public	None	POST /api/registration, /api/payment/verify, /api/login, /api/admin/login, /api/session/**, /api/documents/**, /api/profiles/search, /api/contact
Authenticated	Valid token	GET /api/auth/me
Logged-in (profile)	Profile token (+ code in request)	/api/notifications, /api/interest, /api/profiles/{code}
Admin	ROLE_ADMIN token	GET list/detail endpoints, PUT payment status, screenshot, /api/admin/**

3. Conventions & Errors

Requests and responses use JSON (UTF-8), except the payment upload (multipart/form-data) and the screenshot download (binary image). Timestamps are ISO-8601 local date-time, e.g. 2026-06-27T16:33:13.823.

Error response shape

```
{
  "message": "Human readable error",
  "errors": {           // present only for validation (400) errors
    "personal.firstName": "First name is required"
  }
}
```

Common status codes

Code	Meaning
200 OK	Request succeeded (reads / status update).
201 Created	Resource created (registration / payment).
400 Bad Request	Validation failed; see errors map.
401 Unauthorized	Missing/invalid credentials on a protected endpoint.
403 Forbidden	Authenticated but lacking ROLE_ADMIN.
404 Not Found	Resource id does not exist.
500 Server Error	Unexpected server-side error.

4. Endpoint Summary

Method	Path	Auth	Purpose
POST	/api/registration	Public	Create a registration
POST	/api/registration/temp	Public	Save personal-details draft
POST	/api/login	Public	Profile login (id + email) -> token
POST	/api/admin/login	Public	Admin login -> token
POST	/api/admin/logout	Admin	Invalidate admin token
GET	/api/session/validate	Token	Session liveness (401 if idle/expired)
POST	/api/session/heartbeat	Token	Extend 30-min idle window
POST	/api/session/logout	Token	Invalidate session (profile logout)
GET	/api/profiles/search	Public	Search partner profiles
GET	/api/profiles/{code}	Public	Profile detail (no email/mobile)
POST	/api/contact	Public	Submit a contact message
GET	/api/success-stories	Public	Success-story couples
GET	/api/notifications	Logged-in	User notifications + unread count
POST	/api/notifications/read	Logged-in	Mark all notifications read
POST	/api/interest	Logged-in	Express interest in a profile
POST	/api/interest/respond	Logged-in	Accept / reject an interest
POST	/api/interest/reject	Logged-in	Reject an accepted connection
GET	/api/interest/accepted	Logged-in	Accepted connections (profiles)
GET	/api/messages	Logged-in	Conversation with a connection
POST	/api/messages	Logged-in	Send a message (accepted only)
GET	/api/registration	Admin	List all registrations
GET	/api/registration/{id}	Admin	Get one registration
GET	/api/admin/profiles	Admin	All profiles + details + filters
PUT	/api/admin/profiles/{id}	Admin	Edit a profile (all fields)
PUT	/api/admin/profiles/{id}/payment-status	Admin	Set a profile payment status
GET	/api/admin/contacts	Admin	List contact messages
GET	/api/admin/notifications	Admin	List notifications + recipient mobile
POST	/api/payment/verify	Public	Submit payment + screenshot
PUT	/api/payment/{id}/status	Admin	Verify / reject a payment
GET	/api/payment	Admin	List all payments
GET	/api/payment/{id}	Admin	Get one payment

GET	/api/payment/{id}/screenshot	Admin	Download payment screenshot
POST	/api/documents/upload	Public	Upload a document/image (base64)
GET	/api/documents/profile-image	Public	Get active profile image
GET	/api/auth/me	Auth	Current user + roles

5. Endpoint Details

POST /api/registration

Auth: Public (no authentication)

Creates a matrimonial registration from the 6-step enroll form. The body is a nested JSON object with six sections. `personal.firstName`, `personal.lastName`, `personal.email` and `personal.mobile` are required (email is used for profile login); all other fields are optional strings (`address.sameAsPermanent` is boolean). The response returns a generated `registrationCode` (e.g. REG00001) used as the login ID.

Request body: application/json (nested: `personal`, `horoscope`, `education`, `address`, `family`, `expectation`)

Example request

```
curl -X POST http://localhost:8080/api/registration \
-H "Content-Type: application/json" \
-d '{
  "personal": { "firstName": "Amit", "lastName": "Patil",
    "email": "amit@example.com",
    "mobile": "9876543210",
    "dobDay": "12", "dobMonth": "08", "dobYear": "1995",
    "maritalStatus": "Unmarried" },
  "horoscope": { "rashi": "Mesh", "manglik": "No" },
  "education": { "highestEducation": "B.E", "occupation": "Engineer" },
  "address": { "country": "India", "state": "Maharashtra",
    "city": "Pune", "sameAsPermanent": true },
  "family": { "fatherName": "Ramesh", "familyType": "Nuclear" },
  "expectation": { "ageFrom": "24", "ageTo": "29" }
}'
```

Responses

Code	Body / Meaning
201	{ id, registrationId, registrationCode, message }
400	Validation failed (e.g. missing firstName/lastName/email).

Example response

```
{
  "id": 1,
  "registrationId": "1",
  "registrationCode": "REG00001",
  "message": "Your profile has been submitted successfully! Your login ID is REG00001"
}
```

POST /api/registration/temp

Auth: Public (no authentication)

Saves a DRAFT of the personal-details section when the user clicks "Next" on the first (Personal) step of the enroll form, into the `registrations_temp` table. Upserts by email (repeated Next clicks update the same draft, no duplicates). The draft is deleted automatically once the full profile is created via POST `/api/registration`. Not validated (partial data allowed).

Request body: application/json (only the personal section is used)

Example request

```
curl -X POST http://localhost:8080/api/registration/temp \
-H "Content-Type: application/json" \
-d '{ "personal": { "firstName":"Amit", "lastName":"Patil",
    "email":"amit@example.com", "mobile":"9876543210" } }'
```

Responses

Code	Body / Meaning
200	{ id, message }

Example response

```
{ "id": 1, "message": "Draft saved." }
```

POST /api/login

Auth: Public (no authentication)

Profile login. Validates the Registration ID (the generated registrationCode, e.g. REG00001) and email against the registrations table (email match is case-insensitive). Login succeeds ONLY if the registration's payment has been VERIFIED by an admin; otherwise a pending/rejected error is returned.

Request body: application/json

Field	Type	Req	Description
registrationId	string	Yes	The registrationCode, e.g. REG00001.
email	string	Yes	Email used when registering.

Example request

```
curl -X POST http://localhost:8080/api/login \
-H "Content-Type: application/json" \
-d '{ "registrationId": "REG00001", "email": "amit@example.com" }'
```

Responses

Code	Body / Meaning
200	{ success, id, registrationCode, name, email, maritalStatus, instanceId, token, message }
401	Invalid Registration ID or email.
403	Payment not yet verified by admin (pending), or rejected.
400	Missing/invalid fields.

Example response

```
{
  "success": true,
  "id": 1,
  "registrationCode": "REG00001",
  "name": "Amit Patil",
  "email": "amit@example.com",
  "maritalStatus": "Unmarried",
  "instanceId": "<server-instance-uuid>",
  "token": "9f2c...32hex",
  "message": "Login successful."
}
```

POST /api/admin/login

Auth: Public (no authentication)

Admin login. Validates username + password against the users table (BCrypt) and issues a server-side session token with a 30-minute idle timeout. Send the returned token as the X-Auth-Token header on all admin requests. The instanceId lets the client detect a server restart.

Request body: application/json

Field	Type	Req	Description
username	string	Yes	Admin username (default: admin).
password	string	Yes	Admin password (default: admin123).

Example request

```
curl -X POST http://localhost:8080/api/admin/login \
-H "Content-Type: application/json" \
-d '{ "username": "admin", "password": "admin123" }'
```

Responses

Code	Body / Meaning
200	{ token, username, roles, instanceId }
401	Invalid username or password.
403	Account is not an administrator.

Example response

```
{
  "token": "9f2c1a7b4d...32hex",
  "username": "admin",
  "roles": ["ROLE_ADMIN"],
  "instanceId": "<server-instance-uuid>"
}
```

POST /api/admin/logout

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Invalidates the admin session token on the server immediately.

Example request

```
curl -X POST http://localhost:8080/api/admin/logout \
-H "X-Auth-Token: <token>"
```

Responses

Code	Body / Meaning
204	No content (token invalidated).

GET /api/session/validate

Auth: Token (X-Auth-Token header)

Read-only session liveness check used by both admin and profile clients. Returns 200 while the session is live and 401 once it has idled out (30 min of inactivity) or after a server restart. It does NOT extend the idle window, so periodic polling cannot keep a dead session alive. The client polls this and logs the user out on 401.

Example request

```
curl http://localhost:8080/api/session/validate \  
-H "X-Auth-Token: <token>"
```

Responses

Code	Body / Meaning
200	{ "valid": true }
401	{ "valid": false } - idle-expired, unknown, or restarted.

Example response

```
{ "valid": true }
```

POST /api/session/heartbeat

Auth: Token (X-Auth-Token header)

Records user activity and extends the session's 30-minute idle window. The client calls it on real user interaction (mouse/keyboard/scroll), throttled to about once per minute. Returns 401 if the session has already expired.

Example request

```
curl -X POST http://localhost:8080/api/session/heartbeat \  
-H "X-Auth-Token: <token>"
```

Responses

Code	Body / Meaning
200	{ "valid": true } - window extended.
401	{ "valid": false } - already expired.

Example response

```
{ "valid": true }
```


POST /api/session/logout

Auth: Token (X-Auth-Token header)

Invalidates the current session token on the server. Used by the profile (non-admin) logout so the session does not linger for the rest of the idle window. Idempotent: logging out an unknown/expired token still returns 204.

Example request

```
curl -X POST http://localhost:8080/api/session/logout \
-H "X-Auth-Token: <token>"
```

Responses

Code	Body / Meaning
204	No content (token invalidated).

POST /api/payment/verify

Auth: Public (no authentication)

Submits a payment confirmation with an uploaded screenshot. Sent as multipart/form-data. The screenshot is stored on the server and the record is created with status PENDING_VERIFICATION. registrationId may be a placeholder like "temp-id".

Request body: multipart/form-data

Field	Type	Req	Description
transactionId	text	Yes	UPI transaction id entered by the user.
upId	text	No	UPI id paid to (e.g. babalagin@paytm).
amount	text	No	Amount paid (e.g. 1000).
registrationId	text	No	Related registration id (or "temp-id").
screenshot	file	No*	Payment screenshot image. Max 10 MB. *Expected in practice.

Example request

```
curl -X POST http://localhost:8080/api/payment/verify \
-F "transactionId=TXN123456" \
-F "upiId=babalagin@paytm" \
-F "amount=1000" \
-F "registrationId=1" \
-F "screenshot=@/path/to/screenshot.png;type=image/png"
```

Responses

Code	Body / Meaning
201	{ id, registrationId, transactionId, status, message }

Example response

```
{
  "id": 1,
  "registrationId": "1",
  "transactionId": "TXN123456",
  "status": "PENDING_VERIFICATION",
  "message": "Payment received and pending verification."
}
```

PUT /api/payment/{id}/status

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Admin action to verify or reject a payment. Sets the status, an optional note, and an updatedAt timestamp. status must be one of VERIFIED, REJECTED, PENDING_VERIFICATION.

Request body: application/json

Field	Type	Req	Description
status	string	Yes	VERIFIED REJECTED PENDING_VERIFICATION
note	string	No	Optional admin note explaining the decision.

Example request

```
curl -H "X-Auth-Token: <token>" -X PUT \
  http://localhost:8080/api/payment/1/status \
  -H "Content-Type: application/json" \
  -d '{ "status": "VERIFIED", "note": "Confirmed in bank statement" }'
```

Responses

Code	Body / Meaning
200	Updated Payment object.
400	Invalid status value.
401	Missing/invalid credentials.
403	Not an admin.
404	Payment id not found.

Example response

```
{
  "id": 1, "registrationId": "1", "transactionId": "TXN123456",
  "upiId": "babalagin@paytm", "amount": "1000",
  "status": "VERIFIED", "statusNote": "Confirmed in bank statement",
  "createdAt": "2026-06-27T16:33:13.8", "updatedAt": "2026-06-27T16:37:30.0"
}
```

GET /api/payment

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Returns all payment records, used by the admin dashboard.

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/payment
```

Responses

Code	Body / Meaning
200	Array of Payment objects.
401/403	Not authorized.

Example response

```
[  
  { "id": 1, "transactionId": "TXN123456", "status": "VERIFIED", ... }  
]
```

GET /api/payment/{id}

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)
Returns a single payment by id.

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/payment/1
```

Responses

Code	Body / Meaning
200	Payment object.
404	Not found.

GET /api/payment/{id}/screenshot

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)
Streams the stored screenshot image for a payment. Content-Type matches the uploaded image (e.g. image/png). Because it is protected, fetch it with the Authorization header (an tag will not carry Basic credentials).

Example request

```
curl -H "X-Auth-Token: <token>" \  
http://localhost:8080/api/payment/1/screenshot --output ss.png
```

Responses

Code	Body / Meaning
200	Binary image bytes (image/png, image/jpeg, ...).
404	No screenshot / payment not found.

GET /api/registration

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)
Returns all registration records.

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/registration
```

Responses

Code	Body / Meaning
200	Array of Registration objects.
401/403	Not authorized.

GET /api/registration/{id}

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Returns a single registration by id, including all six nested sections and createdAt.

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/registration/1
```

Responses

Code	Body / Meaning
200	Registration object.
404	Not found.

GET /api/auth/me

Auth: Authenticated (X-Auth-Token header)

Validates the supplied credentials and returns the username and granted roles. Used by the admin login page to confirm a login and check for ROLE_ADMIN.

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/auth/me
```

Responses

Code	Body / Meaning
200	{ username, roles[] }
401	Invalid credentials.

Example response

```
{
  "username": "admin",
  "roles": ["ROLE_ADMIN"]
}
```

GET /api/notifications

Auth: Logged-in profile user

Returns a user notifications (interest received/accepted/rejected, payment updates) newest first, plus the unread count. Shown via the navbar bell after login.

Field	Type	Req	Description
registrationCode	query	Yes	The logged-in user code.

Example request

```
curl "http://localhost:8080/api/notifications?registrationCode=REG000001"
```

Responses

Code	Body / Meaning
200	{ items: [Notification...], unread }

POST /api/notifications/read

Auth: Logged-in profile user

Marks all of a user notifications as read (clears the bell count).

Field	Type	Req	Description
registrationCode	query	Yes	The logged-in user code.

Example request

```
curl -X POST "http://localhost:8080/api/notifications/read?registrationCode=REG000001"
```

Responses

Code	Body / Meaning
200	{ unread: 0 }

POST /api/interest

Auth: Logged-in profile user

Express interest in another profile. Creates an INTEREST_RECEIVED notification for the target.

Request body: application/json

Field	Type	Req	Description
fromCode	string	Yes	Sender code.
toCode	string	Yes	Target code.

Example request

```
curl -X POST http://localhost:8080/api/interest \
  -H "Content-Type: application/json" \
  -d '{ "fromCode": "REG000001", "toCode": "REG000037" }'
```

Responses

Code	Body / Meaning
200	{ message }

POST /api/interest/respond

Auth: Logged-in profile user

Accept or reject a received interest. Notifies the original sender (INTEREST_ACCEPTED / INTEREST_REJECTED).

Request body: application/json

Field	Type	Req	Description
notificationId	string	Yes	The INTEREST_RECEIVED notification id.
action	string	Yes	ACCEPT or REJECT.

Example request

```
curl -X POST http://localhost:8080/api/interest/respond \  
-H "Content-Type: application/json" \  
-d '{ "notificationId": "5", "action": "ACCEPT" }'
```

Responses

Code	Body / Meaning
200	{ message }

POST /api/interest/reject

Auth: Logged-in profile user

Rejects an already-accepted connection. Flips the underlying accepted INTEREST_RECEIVED notification(s) between the two profiles (either direction) to REJECTED, so they are no longer connected (contact details are hidden again), and notifies the other party. Used by the "Reject" button on the Accepted Profiles page.

Request body: application/json

Field	Type	Req	Description
fromCode	string	Yes	The viewer (logged-in user) registration code.
code	string	Yes	The connected profile to reject.

Example request

```
curl -X POST http://localhost:8080/api/interest/reject \  
-H "Content-Type: application/json" \  
-d '{ "fromCode": "REG00037", "code": "REG00041" }'
```

Responses

Code	Body / Meaning
200	{ rejected: true false, message }

Example response

```
{ "rejected": true, "message": "Connection rejected." }
```

GET /api/messages

Auth: Logged-in profile user (accepted connection)

Returns the full conversation between two profiles (oldest first). Allowed only when the two are accepted-

connected, otherwise 403.

Field	Type	Req	Description
code	query	Yes	The logged-in user code.
withCode	query	Yes	The connected profile code.

Example request

```
curl "http://localhost:8080/api/messages?code=REG00041&withCode=REG00042"
```

Responses

Code	Body / Meaning
200	[{ id, fromCode, toCode, content, createdAt }, ...]
403	Not an accepted connection.

Example response

```
[
  { "id":1, "fromCode":"REG00041", "toCode":"REG00042",
    "content":"Hello from A", "createdAt":"2026-06-28T23:16:53" }
]
```

POST /api/messages

Auth: Logged-in profile user (accepted connection)

Sends an in-app message to a connected profile. Allowed only between accepted connections (403 otherwise). Contact details (email/mobile) are never shared, so connected users communicate here. A bell notification of type MESSAGE is created for the recipient.

Request body: application/json

Field	Type	Req	Description
fromCode	string	Yes	Sender registration code.
toCode	string	Yes	Recipient registration code.
content	string	Yes	Message text (non-empty).

Example request

```
curl -X POST http://localhost:8080/api/messages \
-H "Content-Type: application/json" \
-d '{ "fromCode":"REG00041", "toCode":"REG00042", "content":"Hello" }'
```

Responses

Code	Body / Meaning
200	{ id, fromCode, toCode, content, createdAt }
400	Missing fields / empty content.
403	Not an accepted connection.

Example response

```
{ "id":1, "fromCode":"REG00041", "toCode":"REG00042",  
  "content":"Hello", "createdAt":"2026-06-28T23:16:53" }
```

PUT /api/admin/profiles/{id}/payment-status

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Sets the payment status for a registration from the admin Profiles edit page. Updates the latest payment, or creates one if none exists; notifies the user.

Field	Type	Req	Description
id	path	Yes	Registration id.
status	query	Yes	VERIFIED PENDING_VERIFICATION REJECTED.

Example request

```
curl -H "X-Auth-Token: <token>" -X PUT "http://localhost:8080/api/admin/profiles/33/payment-status?status=VERIFIED"
```

Responses

Code	Body / Meaning
200	{ registration, paymentStatus }
404	Registration not found.

GET /api/profiles/{code}

Auth: Public

Full profile detail by registration code, used by the "View Profile" popup. Email & mobile are NEVER returned (even to an accepted connection) -- connected users communicate via the in-app messaging API instead.

Field	Type	Req	Description
code	path	Yes	Registration code, e.g. REG00033.
viewerCode	query	No	Logged-in viewer code (used for the connection/accepted check). Email/mobile are never returned regardless.

Example request

```
curl "http://localhost:8080/api/profiles/REG00037?viewerCode=REG00065"
```

Responses

Code	Body / Meaning
200	Full Registration. personal.email/mobile only present for an accepted connection.
404	No profile with that code.

GET /api/success-stories

Auth: Public

Returns success-story couples: registrations flagged successStory=Y, each paired with its partner (resolved via partnerId). Couples flagged on both sides are de-duplicated.

Example request

```
curl http://localhost:8080/api/success-stories
```

Responses

Code	Body / Meaning
200	Array of { person: ProfileSummary, partner: ProfileSummary null }

Example response

```
[
  { "person": { "registrationCode": "REG00033", "name": "Sai Joshi", ... },
    "partner": { "registrationCode": "REG00038", "name": "Rahul Joshi", ... } }
]
```

POST /api/contact

Auth: Public

Saves a "Contact Us" form submission to the contact_messages table.

Request body: application/json

Field	Type	Req	Description
name	string	Yes	Full name.
phone	string	Yes	Phone number.
email	string	Yes	Valid email.
subject	string	Yes	e.g. general / registration / complaint.
message	string	Yes	Message body.

Example request

```
curl -X POST http://localhost:8080/api/contact \
-H "Content-Type: application/json" \
-d '{ "name": "Ravi Kumar", "phone": "+91 98765 43210",
      "email": "ravi@example.com", "subject": "general",
      "message": "I have a question." }'
```

Responses

Code	Body / Meaning
201	{ id, message }
400	Validation failed (missing/invalid fields).

Example response

```
{
  "id": 1,
  "message": "Thank you! Your message has been received."
}
```

GET /api/admin/contacts

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Lists all Contact-Us submissions (newest first) for the admin Contact Messages page.

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/admin/contacts
```

Responses

Code	Body / Meaning
200	Array of ContactMessage objects.
401/403	Not an admin.

GET /api/admin/notifications

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Lists every notification (newest first) with the recipient's name and mobile, for the admin Notifications page. The UI builds a wa.me "Send on WhatsApp" click-to-send link per row (manual send; no WhatsApp API integration).

Example request

```
curl -H "X-Auth-Token: <token>" http://localhost:8080/api/admin/notifications
```

Responses

Code	Body / Meaning
200	[{ id, registrationCode, recipientName, recipientMobile, type, message, fromName, status, read, createdAt }, ...]
401/403	Not an admin.

GET /api/profiles/search

Auth: Public

Searches registered profiles for the "Find Your Life Partner" form. lookingFor matches the profile gender ("Groom"/"Bride"). Age is matched as a range; education/location/community are exact matches (the value "Any" or blank means no filter on that field).

Field	Type	Req	Description
lookingFor	query	No	Groom Bride (matches profile gender).
maritalStatus	query	No	e.g. Unmarried / Divorced.
ageFrom / ageTo	query	No	Age range: age >= ageFrom and age <= ageTo.
heightFromInches / heightToInches	query	No	Height range in total inches (feet*12+inch).
mangalik	query	No	Yes to return only mangalik profiles.
education	query	No	e.g. Graduate / Post Graduate.
occupationType	query	No	e.g. Private Service / Business / Professional.

incomeRange	query	No	e.g. 8 - 16 Lakh.
location	query	No	City, e.g. Pune.
nativePlace	query	No	Native place city.
community	query	No	e.g. Marathi / Maratha.

Example request

```
curl "http://localhost:8080/api/profiles/search?
lookingFor=Bride&ageFrom=22&ageTo=30&heightFromInches=60&heightToInches=66&location=Pune"
```

Responses

Code	Body / Meaning
200	Array of ProfileSummary objects (see Data Models).

Example response

```
[
  { "registrationCode": "REG000002", "name": "Sai Joshi", "gender": "Bride",
    "age": 25, "education": "Graduate", "occupation": "Teacher",
    "city": "Pune", "community": "Marathi", "maritalStatus": "Unmarried" }
]
```

POST /api/documents/upload

Auth: Public (multipart/form-data)

Uploads a document/image which is stored base64-encoded in the documents table. Used to set a user profile photo. Multiple uploads are retained; the latest of a docType is the active one.

Request body: multipart/form-data

Field	Type	Req	Description
registrationCode	text	Yes	Owner, e.g. REG000001.
docType	text	No	Category; defaults to "profileImage".
file	file	Yes	The file (image). Max 10 MB.

Example request

```
curl -X POST http://localhost:8080/api/documents/upload \
-F "registrationCode=REG000001" \
-F "docType=profileImage" \
-F "file=@/path/to/photo.png;type=image/png"
```

Responses

Code	Body / Meaning
201	{ id, registrationCode, docType, fileName, contentType, dataUrl, createdAt }

Example response

```
{
  "id": 1, "registrationCode": "REG00001", "docType": "profileImage",
  "fileName": "photo.png", "contentType": "image/png",
  "dataUrl": "...",
  "createdAt": "2026-06-27T19:21:00"
}
```

GET /api/documents/profile-image

Auth: Public

Returns the active (latest) profileImage document for a user as a ready-to-use data URL, or 404 if none exists.

Field	Type	Req	Description
registrationCode	query	Yes	e.g. REG00001.

Example request

```
curl "http://localhost:8080/api/documents/profile-image?registrationCode=REG00001"
```

Responses

Code	Body / Meaning
200	{ id, registrationCode, docType, fileName, contentType, dataUrl, createdAt }
404	No profile image uploaded yet.

GET /api/admin/profiles

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Returns all registrations with full details plus each one's current payment status. Powers the admin "Profiles" page. All filters are optional.

Field	Type	Req	Description
gender	query	No	Groom or Bride.
isActive	query	No	Y or N.
createdFrom	query	No	Created date >= (yyyy-MM-dd).
createdTo	query	No	Created date <= (yyyy-MM-dd).
paymentStatus	query	No	VERIFIED PENDING_VERIFICATION REJECTED NONE.

Example request

```
curl -H "X-Auth-Token: <token>" "http://localhost:8080/api/admin/profiles?gender=Bride&isActive=Y&paymentStatus=VERIFIED"
```

Responses

Code	Body / Meaning
200	Array of { registration: {...all fields...}, paymentStatus }.
401/403	Not an admin.

Example response

```
[
  { "registration": { "registrationCode": "REG00001", "personal": {...},
    "isActive": "Y", "successStory": "N", "createdDate": "..."},
    "paymentStatus": "VERIFIED" }
]
```

PUT /api/admin/profiles/{id}

Auth: Admin (X-Auth-Token header, ROLE_ADMIN)

Admin edit. Overwrites all editable fields of the registration with the supplied body (the full Registration JSON). The id is fixed (path), and createdAt/createdDate are preserved; updatedAt is refreshed automatically.

Request body: application/json (full Registration object)

Example request

```
curl -H "X-Auth-Token: <token>" -X PUT http://localhost:8080/api/admin/profiles/33 \
-H "Content-Type: application/json" \
-d '{ "registrationCode": "REG00033", "isActive": "N",
  "personal": { "firstName": "Sai", "lastName": "Joshi", ... },
  "address": { "city": "Mumbai", ... }, ... }'
```

Responses

Code	Body / Meaning
200	{ registration: {...updated...}, paymentStatus }
404	Registration id not found.
401/403	Not an admin.

6. Data Models

Registration

id (number), registrationCode (string, e.g. REG00001), createdAt/createdDate/updatedAt (timestamps), isActive (Y/N, default Y), successStory (Y/N, default N), partnerId (string, max 20), and six nested section objects below.

personal

Field	Type	Notes
firstName	string	Required
middleName	string	
lastName	string	Required
email	string	Required - used for login
mobile	string	Required - 10-digit mobile
gender	string	Groom / Bride (used by search)
community	string	e.g. Marathi (used by search)
dobDay / dobMonth / dobYear	string	Date of birth parts

subCaste	string	
maritalStatus	string	e.g. Unmarried / Divorced
heightFeet / heightInches	string	
weight	string	
bloodGroup	string	
complexion	string	
physicalDisability	string	Yes / No
disabilityDetails	string	
diet	string	e.g. Veg / Non-Veg
spectacles	string	Yes / No

horoscope

Field	Type	Notes
rashi	string	
nakshatra	string	
charan	string	
gan	string	
gotra	string	
manglik	string	Yes / No
birthTime	string	
birthPlace	string	

education

Field	Type	Notes
highestEducation	string	
fieldOfStudy	string	
institution	string	
occupation	string	
employer	string	
annualIncome	string	

address

Field	Type	Notes
country	string	Defaults to India
state	string	
city	string	
pincode	string	

permanentAddress	string	
currentAddress	string	
sameAsPermanent	boolean	true / false

family

Field	Type	Notes
fatherName	string	
fatherOccupation	string	
motherName	string	
motherOccupation	string	
siblings	string	
familyType	string	e.g. Nuclear / Joint
familyStatus	string	

expectation

Field	Type	Notes
ageFrom / ageTo	string	Preferred age range
heightFrom / heightTo	string	Preferred height range
education	string	
occupation	string	
location	string	
otherPreferences	string	

Payment

Field	Type	Notes
id	number	Generated id
registrationId	string	Related registration (or "temp-id")
transactionId	string	UPI transaction id
upId	string	
amount	string	
screenshotPath	string	Server file path (admin reads only)
screenshotFilename	string	Original uploaded filename
screenshotContentType	string	e.g. image/png
status	string	PENDING_VERIFICATION VERIFIED REJECTED
statusNote	string	Admin note (nullable)
createdAt	timestamp	When submitted
updatedAt	timestamp	When status last changed (nullable)

AuthMe

Field	Type	Notes
username	string	Authenticated username
roles	string[]	e.g. ["ROLE_ADMIN"]

ProfileSummary (search result)

Field	Type	Notes
registrationCode	string	e.g. REG00002
name	string	First + last name
gender	string	Groom / Bride
age	number	Computed from dobYear
height	string	e.g. 5'6" (from total inches)
mangalik	string	Yes / No
education	string	
occupation	string	
occupationType	string	e.g. Private Service
incomeRange	string	e.g. 8 - 16 Lakh
city	string	
nativePlace	string	
community	string	
maritalStatus	string	

Notification

Field	Type	Notes
id	number	Generated id
registrationCode	string	Recipient
type	string	INTEREST_RECEIVED / INTEREST_ACCEPTED / INTEREST_REJECTED / PAYMENT
message	string	
fromCode / fromName	string	Who triggered (interest)
status	string	PENDING/ACCEPTED/REJECTED (interest received)
read	boolean	Read flag
createdDate	timestamp	

ContactMessage

Field	Type	Notes
-------	------	-------

id	number	Generated id
fullName	string	Sender name
phone	string	
email	string	
subject	string	
message	string	Up to 2000 chars
createdDate	timestamp	When submitted

Document

Field	Type	Notes
id	number	Generated id
registrationCode	string	Owner, e.g. REG00001
docType	string	e.g. profileImage
fileName	string	Original filename
contentType	string	e.g. image/png
dataUrl	string	data:<type>;base64,<...> (built from stored base64)
createdAt	timestamp	Upload time